Bilkent University Electrical and Electronics Department

EE102-01 Lab 2 Report:

Introduction to VHDL

Muhammed Mesud Bilirgen – 22301965 – 17th Feb 2025

Purpose

The aim of the lab is to understand errors and warning in Vivado while debugging a combinational circuit on VHDL. After debugging, realizing the circuit on BASYS3 board and simulating the code by writing a test bench are amongst purposes.

Methodology

In the first task, the logic gates in submodule1 are changed according to last digit of ID. In the second task, bugs are fixed by examining errors and warnings. In the third task, debugged circuit is generated on BASYS3 board. In the fourth task, we generated a waveform of outputs by using a test bench code we have written. In the fifth task, we composed and compared three different schematic types.

Results

- In VHDL, a module is defined inside an ENTITY declaration by specifying signals acting as inputs and outputs using the PORT keyword.
- Component is a reusable VHDL module using component declaration of the VHDL code. PORT MAP allows modular design by connecting signals from top module to the input and output ports of the instantiated module.
- Constrain file is a statement of how logical signals are matched to physical FPGA pins. Therefore, it is essential to realize the code on board.
- A testbench makes the code simulate all possible inputs and observe outputs, respectively. Hence, it is used to compose a waveform to verify the functionality of a module.

Task 1: The logic gates in submodule1.vhd lines 14, 15, and 16 changed to NAND XOR XNOR, as the last digit of my ID which is 5 required.

```
11 architecture Structural_sub1 of sub_module1 is
12
13 begin
14     o_output_byte(0)          <= i_input_byte(0) nand i_input_byte(1); --CHANGED: Change these three operators according to the table
15     o_output_byte(1)          <= i_input_byte(2) xor i_input_byte(3);
16     o_output_byte(2)          <= i_input_byte(4) xnor i_input_byte(5);
17     o_output_byte(5 downto 3) <= "010";
18     o_output_byte(7 downto 6) <= (others => '0');
19
20 end Structural_sub1;
```

Figure 1: Converted logic gates

Task 2: By examining codes, errors, warning, bugs are noticed and eliminated.

1) Due to the forgotten "}" typo in line 28 of the constraint file, Vivado cannot read the code which has opened but not closed curly bracket.

```
set_property PACKAGE_PIN U19 [get_ports {o_LED[2]]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}]
set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[3]}]
```

[Designutils 20-970] Unrecognized or unsupported command 'set_property PACKAGE_PIN U19 [get_ports {o_LED[2]] set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}] set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}] set_property IOSTANDARD LVCMOS33 [get_por ... (truncated) ' found in constraint file. [constraints_basys3.xdc:28]

Figure 2.1: The typo "}" and its warning

2) Module 2 as called in the module file sub_module2_the_beast is called sub_module2 in the top_module so inconsistent naming cause Vivado to not recognize it. Changing all "sub_module2" to "sub_module2_the_beast" eliminates the bug.

```
    component sub_module2 is
        port (
            i_switch_inputs : in  STD_LOGIC_VECTOR (7 downto 0);
            o_output_vector : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component sub_module2;

    signal s_output_1, s_output_2 : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
    signal s_output_3             : unsigned(7 downto 0)         := (others => '0');

begin

    sub_module1_2 : sub_module1
        port map (
            i_input_byte  => i_SW,
            o_output_byte => s_ouput_1
        );

    sub_module2_1 : sub_module2
        port map (s_output_2, i_SW);
    s_output_3 <= unsigned(s_output_2) + 25;

    o_LED <= (not std_logic_vector(s_output_3)) xor s_output_1;

end Structural_top;
```

Figure 2.2: The typo "}" and its warning

3) There was a typo in sub_module1_2 which is s_ouput_1. Therefore Vivado throws the error for s_ouput_1 which is not declared. Debugging is done by fixing that typo.

```
    sub_module1_2 : sub_module1
        port map (
            i_input_byte  => i_SW,
            o_output_byte => s_ouput_1
        );
```

[Synth 8-36] 's_ouput_1' is not declared [top_module.vhd:36]

Figure 2.3: The typo "s_ouput_1" and its error

4) There was a mistake in positional port map declaration which causes “i_sw” to match with output of module. Therefore, Vivado throws the error for mode of “i_sw” which is “in”. It is fixed by changing the order of positional port map.

```
39      sub_module2_1 : sub_module2_the_beast
40          port map (s_output_2, i_SW);
41      s_output_3 <= unsigned(s_output_2) + 25;
```

[Synth 8-8985] 'i_sw' with mode 'in' cannot be updated [top_module.vhd:40]

Figure 2.4: The misused positional port map and its error

5) The device chosen in project settings is not BASYS3 board. Since it does not acknowledge the port names in the constraint file such as “W16”,”W17”,”W15”; the code cannot be uploaded to BASYS3. Adjusting part number to BASYS3’s (xc7a35tcpg236-1) from the project manager solved this problem.

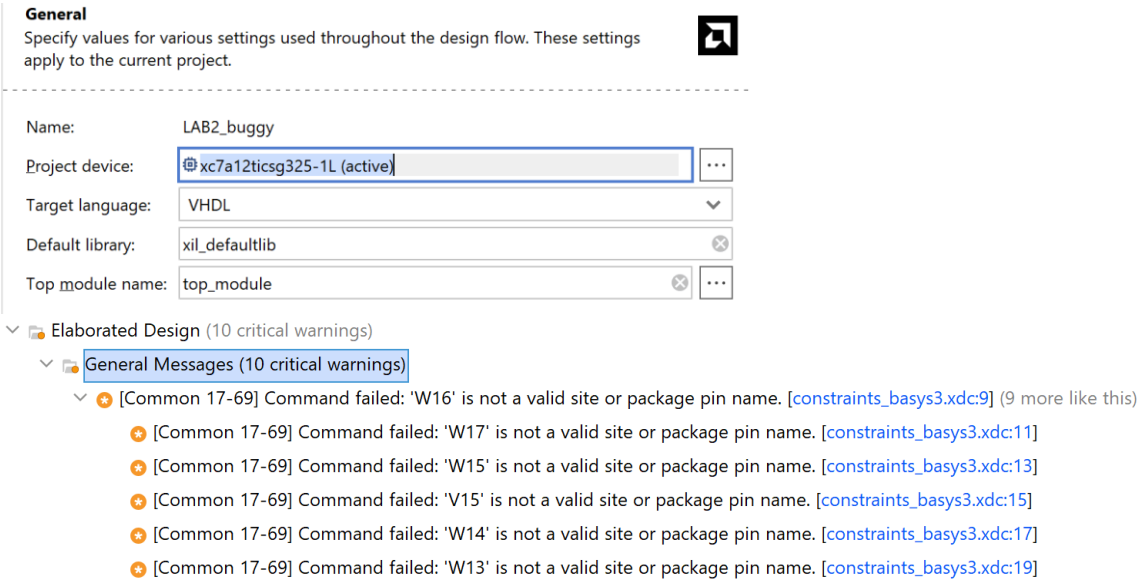

Figure 2.5: Entered project device which is not BASYS3 and its warning

6) Actually, sixth bug does not affect the simulations or running of the code. Thus, there is no any error or warning. Only problem it causes is we cannot observe expected results on BASYS3. Adjusting the order of V14 and U16 LEDs in the constraint file solves the problem.

```
# LEDs
set_property PACKAGE_PIN V14 [get_ports {o_LED[0]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[0]}]
set_property PACKAGE_PIN E19 [get_ports {o_LED[1]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[1]}]
set_property PACKAGE_PIN U19 [get_ports {o_LED[2]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}]
set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[3]}]
set_property PACKAGE_PIN W18 [get_ports {o_LED[4]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[4]}]
set_property PACKAGE_PIN U15 [get_ports {o_LED[5]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[5]}]
set_property PACKAGE_PIN U14 [get_ports {o_LED[6]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[6]}]
set_property PACKAGE_PIN U16 [get_ports {o_LED[7]}]
    set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[7]}]
```

Figure 2.6: Wrong ordered LEDs

Task 3: The Basys-3 board is programmed with the generated bit stream code. As declared in the constraint file, inputs are determined by switches and outputs observed by LEDs on the board. Observed functionality of my circuit is as examples shown in Figure 3.1-3.6 with the form “f(<input>) = <output>”.

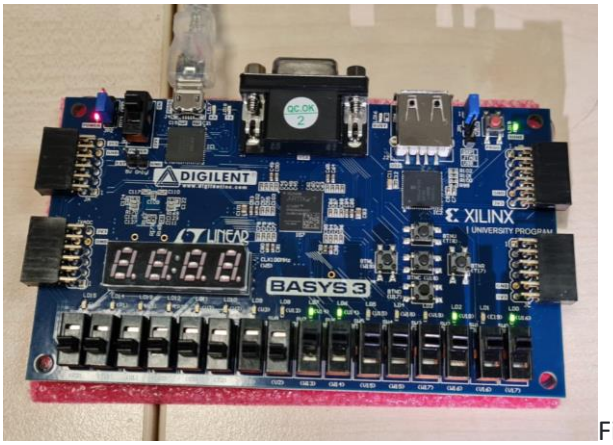

Figure 3.1: f(1111111) =10100011

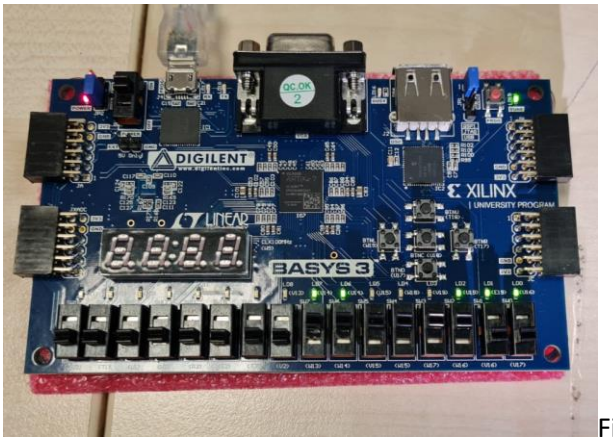

Figure 3.2: f(00111111) = 11100011

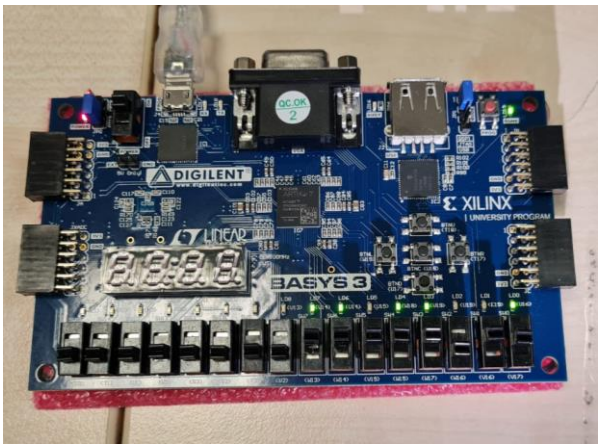

Figure 3.3: f(11011111) =10011011

Figure3.4: f(11011011)=10000011

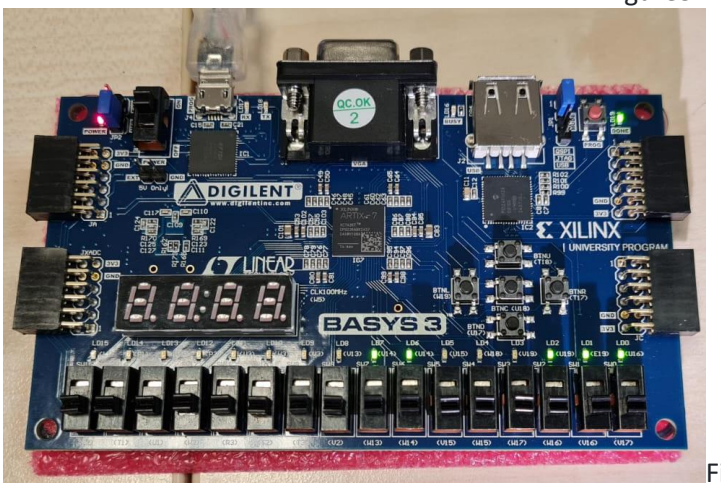

Figure 3.5: f(00000000)=11100011

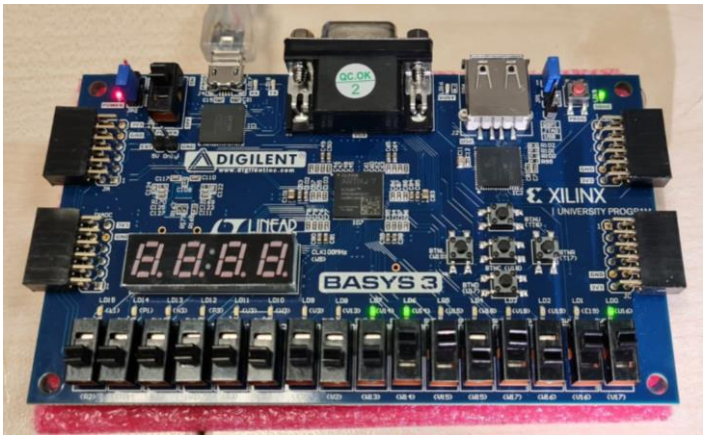

Figure 3.6: f(11010110)=10000011

Task 4: To investigate the code working correctly or not, a test bench code, which shows all possible input configurations and their inputs, is written (Appendices 4).

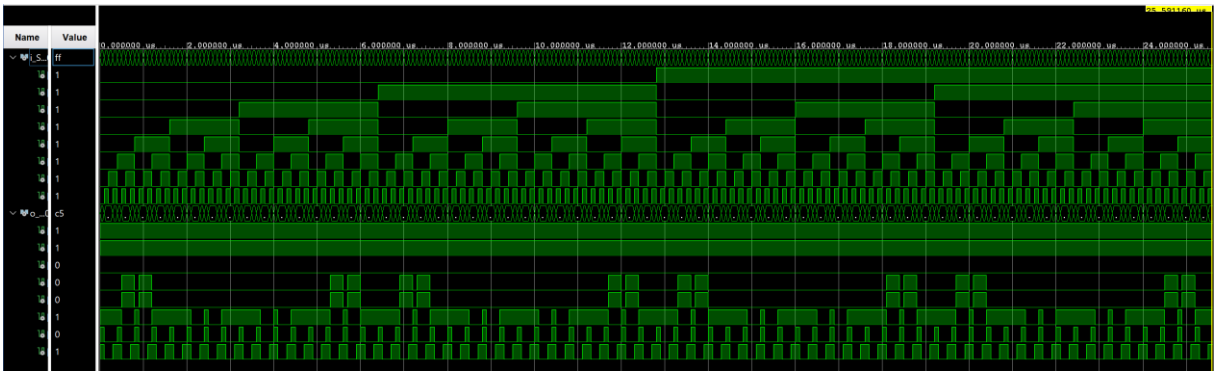

Figure 4: The waveform

Task 5: All three schematics Vivado gives for our code are as shown at Figure 5.1-4. In the RTL schematics, the circuit are shown with the logic gates and arithmetic operations as declared with modules. On the other hand, synthesis and implementation schematics utilize LUTs (look-up-table) rather than logic gates and arithmetic operations. Therefore, they serve as a representation of how the FPGA will realize the circuit. While RTL schematic is convenient to see how user planned to realize the aim of the circuit, synthesis and implementation schematics show how the circuit does this. It is noticeable that implemented and synthetized design o_LED[7] and o_LED[6] must be always 1. Similarly, o_LED[5] must be 0, o_LED[3] and o_LED[4] must be same, i_SW[6] and i_SW[7] must not affect the outputs. It is observable that these results correlate with the waveform results from test.

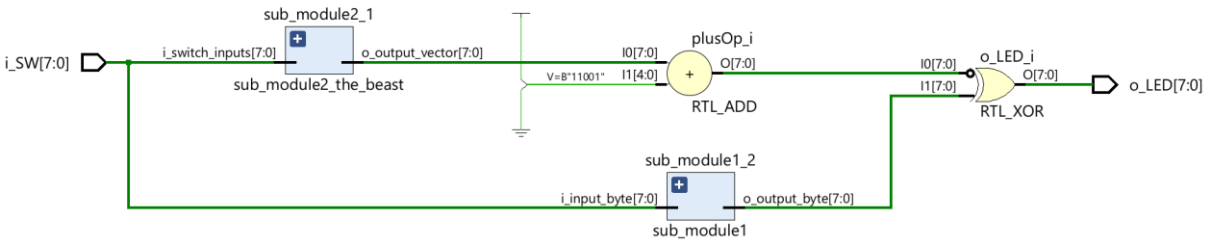

Figure 5.1: The RTL schematic

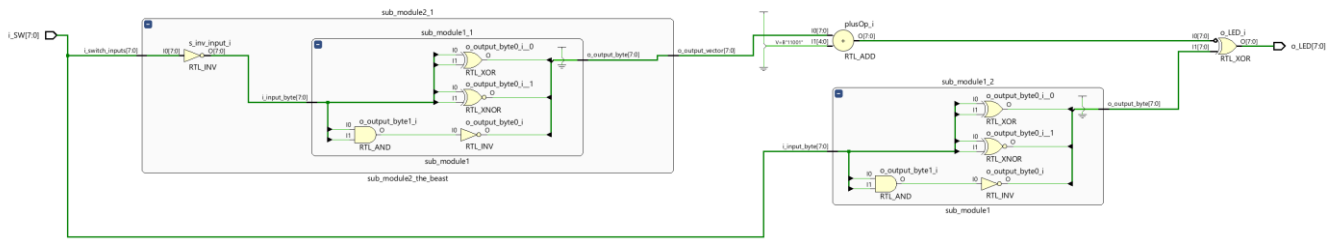

Figure 5.2: The RTL schematic (opened)

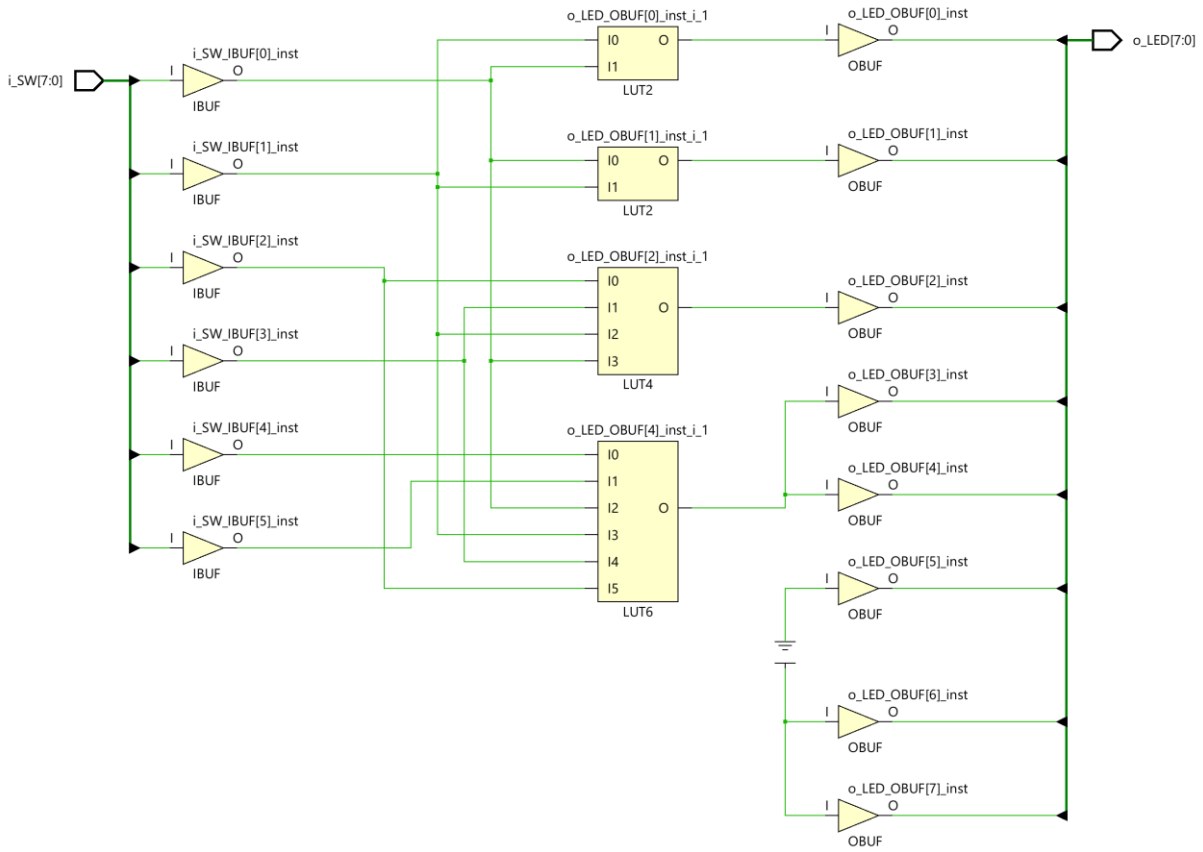

Figure 5.3: The synthesis schematic

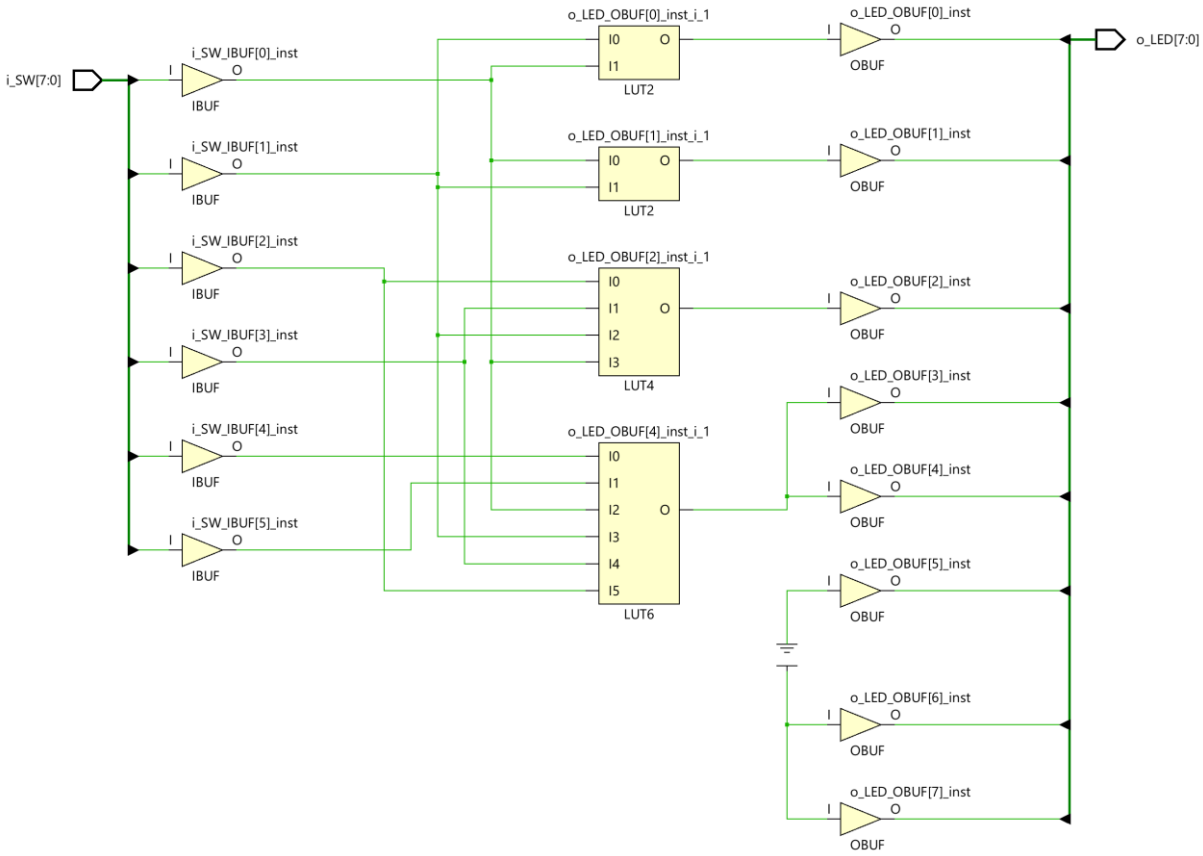

Figure 5.4: The implementation schematic

Conclusion

The experiment is successful to fulfil the aim of the lab which is to get used to VHDL and Vivado. While solving problems we encountered on debugging, we are able to learn some functionality of Vivado and their importance. Encountering logic gates and truth tables helped our understanding of recent lectures of EE102. Moreover, writing a test bench code allow us to gain some insight into some functions and a sequential VHDL code.

Appendices

The modified codes:

1) **sub_module1.vhd**

```
architecture Structural_sub1 of sub_module1 is

begin
	o_output_byte(0)	<= i_input_byte(0) nand i_input_byte(1); --CHANGED: Change these three operators according to the table in the lab document
	o_output_byte(1)	<= i_input_byte(2) xor i_input_byte(3);
	o_output_byte(2)	<= i_input_byte(4) xnor i_input_byte(5);
	o_output_byte(5 downto 3) <= "010";
	o_output_byte(7 downto 6) <= (others => '0');

end Structural_sub1;
```

2) **constraints_basys3.xdc**

```
# LEDs
set_property PACKAGE_PIN U16 [get_ports {o_LED[0]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[0]}]
set_property PACKAGE_PIN E19 [get_ports {o_LED[1]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[1]}]
set_property PACKAGE_PIN U19 [get_ports {o_LED[2]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[2]}]
set_property PACKAGE_PIN V19 [get_ports {o_LED[3]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[3]}]
set_property PACKAGE_PIN W18 [get_ports {o_LED[4]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[4]}]
set_property PACKAGE_PIN U15 [get_ports {o_LED[5]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[5]}]
set_property PACKAGE_PIN U14 [get_ports {o_LED[6]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[6]}]
set_property PACKAGE_PIN V14 [get_ports {o_LED[7]}]
	set_property IOSTANDARD LVCMOS33 [get_ports {o_LED[7]}]
```

3) **top_module.vhd**

```
  component sub_module2_the_beast is
    port (
```

```
        i_switch_inputs : in  STD_LOGIC_VECTOR (7 downto 0);
        o_output_vector : out STD_LOGIC_VECTOR (7 downto 0)
      );
   end component sub_module2_the_beast;

   signal s_output_1, s_output_2 : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');
   signal s_output_3            : unsigned(7 downto 0)        := (others => '0');

begin

   sub_module1_2 : sub_module1
      port map (
        i_input_byte  => i_SW,
        o_output_byte => s_output_1
      );

   sub_module2_1 : sub_module2_the_beast
      port map (i_SW , s_output_2);
   s_output_3 <= unsigned(s_output_2) + 25;

   o_LED <= (not std_logic_vector(s_output_3)) xor s_output_1;

end Structural_top;
```

4) testBenchLab2.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity testBenchLab2 is
--  A testbench does not need ports since it drives and monitors internal signals within the simulation.
end testBenchLab2;

architecture Behavioral of testBenchLab2 is
   component top_module is
   port(
    i_SW : in STD_LOGIC_VECTOR (7 downto 0);
    o_LED : out STD_LOGIC_VECTOR (7 downto 0)
   );
   end component;
   signal i_SW : std_logic_vector(7 downto 0);
   signal o_LED : std_logic_vector(7 downto 0);
   begin
   UUT : top_module port map(i_SW ,o_LED);
   STIM_PROCESS : process begin
   for i in 0 to 255 loop
    i_SW <= std_logic_vector(to_unsigned(i, 8));
    wait for 100 ns;
   end loop;
   assert false report "SIMULATION FINISHED" severity failure;
   end process;
```

```
end Behavioral;
```